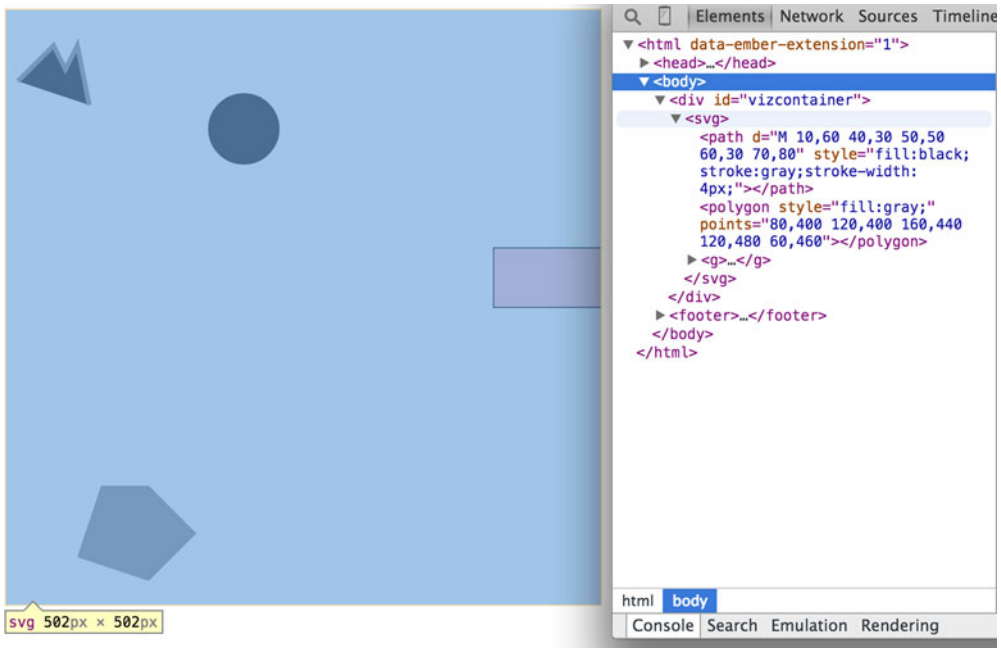


Figure 1.17 Inspecting the DOM of a web page with an SVG canvas reveals the nested graphical elements as well as the style and attributes that determine their position. Notice that the circle and rectangle exist as child elements of a group.

traditional JavaScript selectors like `document.getElementById` or with D3, removing them or changing the style like so:

```
d3.select("circle").remove()
d3.select("rect").style("fill", "purple")
```

Deletes the circle

Changes the rectangle color to purple

Now refresh your page and let's take a look at the new elements. You're familiar with divs, and it's useful to put an SVG canvas in a div so you can access the parent container for layout and styling. Let's take a look at each of the elements we've added.

<SVG>

This is your canvas on which everything is drawn. The top-left corner is 0,0, and the canvas clips anything drawn beyond its defined height and width of 500,500 (the rectangle in our example). An `<svg>` element can be styled with CSS to have different borders and backgrounds. The `<svg>` element can also be dynamically resized using the `viewBox` attribute, which is more complex and beyond the scope of the overview here.

You can use CSS (which we'll touch on later in this chapter) to style your SVG canvas or use D3 to add inline styles like so: